

A Configurable Software Model of a Self-Adaptive Robotic System

Juliane Päßler^{a,*}, Maurice H. ter Beek^b, Ferruccio Damiani^c, Einar Broch Johnsen^a,
S. Lizeth Tapia Tarifa^a

^a*University of Oslo, Gaustadalléen 23B, NO-0373, Oslo, Norway*

^b*CNR-ISTI, Via Giuseppe Moruzzi 1, Pisa, I-56124, Italy*

^c*University of Turin, Corso Svizzera 185, I-10149, Torino, Italy*

Abstract

Self-adaptation, meant to increase reliability, is a crucial feature of cyber-physical systems operating in uncertain physical environments. Ensuring safety properties of self-adaptive systems is of utter importance, especially when operating in remote environments where communication with a human operator is limited, like under water or in space. This paper presents a software model that allows the analysis of one such self-adaptive system, a configurable underwater robot used for pipeline inspection, by means of the probabilistic model checker ProFeat. Furthermore, it shows that the configurable software model is easily extensible to further, possibly more complex use cases and analyses.

Keywords: Self-Adaptive Systems, Dynamic Software Product Lines, Probabilistic Model Checking, ProFeat

Table 1: Code Metadata

Code metadata description	
Current code version	v1.1.1
Permanent link to code/repository used for this code version	https://github.com/remaro-network/auv_profeat/releases/tag/ifm2023
Permanent link to reproduce capsule	https://doi.org/10.5281/zenodo.10090486
Legal code license	Apache License 2.0
Coder versioning system used	git
Software code languages, tools and services used	PRISM input and property language, ProFeat, iFM23 virtual machine
Compilation requirements, operating environments and dependencies	Linux, Windows, MacOS (Intel for capsule)
If available, link to developer documentation/manual	https://github.com/remaro-network/auv_profeat/blob/ifm_artifact/README.md
Support email for questions	julipas@uio.no

1. Motivation and Significance

Self-Adaptive Systems (SASs) often operate in dangerous and dynamic environments where human supervision is limited or impossible, like under water or in space. Therefore, it is important to ensure that safety properties are maintained by the system throughout system operation. Once in operation, SASs are frequently reconfigured, which often means

*Corresponding author

switching between different system configurations during runtime. The analysis of all these different yet partially redundant configurations separately is a tedious, time-consuming, and error-prone task, especially because this ignores the changes between configurations.

In Päßler et al. [20], we showed the advantages of modelling such an SAS as a family of systems, where each family member corresponds to a possible configuration, which allows for family-based modelling and analysis as a means to combat redundancy [25]. To do so, we used formal models and tools from the field of Software Product Lines (SPL) [2]. We also used the fact that SASs can be implemented using a two-layered approach, decomposing the system into a *managed* and a *managing* subsystem [15, 27], with the managed subsystem implementing the domain concerns (e.g., navigating a robot to a specific position) and the managing subsystem implementing the adaptation logic (e.g., reconfiguring due to changing environmental conditions). This separation of concerns is catered for by ProFeat [9], a tool for probabilistic family-based model checking. ProFeat provides a means to simultaneously analyse, in one single run, a family of models, each corresponding to a valid configuration.

This paper contributes a configurable software model of a self-adaptive robotic system, namely an Autonomous Underwater Vehicle (AUV) used to search for and follow a pipeline located on a seabed. Furthermore, the paper illustrates how to perform analyses of such models with ProFeat, and how to modify and extend the model. The model has been used for a case study, presented in Päßler et al. [20, 21], and is inspired by the exemplar SUAVE [22]. In contrast to Päßler et al. [20], this paper does not detail the software model. Instead, it shows with the software model, how an existing framework for modelling and analysing family-based systems can be used for SAS research. Furthermore, it shows how the software model can be extended for further, possibly more complex SAS models and analyses.

2. Software Description

Our configurable software model is built for analysis with the family-based model checker ProFeat. ProFeat¹ is a tool that extends the probabilistic model checker PRISM² [16] with functionalities such as family models, features, and feature switches, thus enabling family-based modelling and analysis of probabilistic systems in which feature configurations may dynamically change during runtime. ProFeat translates its input to the input language of PRISM to use PRISM’s reasoning engine for probabilistic (family-based) model checking.

Akin to SASs, an input model of ProFeat can be seen as a two-layered model in which the behaviour of a family of systems that differ in their feature configurations, as defined by a *feature model* that specifies the features and their relations and constraints, is specified as *feature modules* (i.e., the ‘managed’ behavioural model) along with a *feature controller* that can activate and deactivate features at runtime (i.e., the ‘managing’ behavioural model), thus changing (reconfiguring) system behaviour. Furthermore, possible environments can be specified as separate modules that interact with the modules of the behavioural models.

The software model of the AUV case study in Päßler et al. [21] contains the following ProFeat modules:

¹<https://pchrzon.github.io/profeat/>

²<https://www.prismmodelchecker.org/manual>

- a feature model, defining the functionalities of the AUV as features and specifying their relations and constraints as well as feature-specific costs implemented as *rewards* (e.g., time and energy);
- a probabilistic, feature-guarded model of the managed subsystem (i.e., a probabilistic featured transition system [11]), defining the behaviour of the different configurations of the SAS as well as the possible switches between them (i.e., reconfigurations);
- a probabilistic model of the environment (i.e., water visibility);
- a feature controller, representing the managing subsystem of the SAS, that activates and deactivates features of the feature model during runtime (while satisfying the constraints of the feature model), based on both environmental and internal conditions, thereby enabling and disabling specific configurations and behaviour of the managed subsystem;
- properties concerning expected rewards (a.k.a. costs) and probabilities.

It is also possible to specify more modules that interact with the already existing ones, like, e.g., a model of how the hardware of the managed subsystem fails or a model of the battery consumption of the AUV. The case study’s repository³ contains one such extension by including two sensors for vision (i.e., a camera and a (side-scan) sonar) and a separate hardware module that models how these sensors can fail (permanently) or get blocked (in case of the camera, e.g., due to natural or human waste sticking to it) at runtime, causing the need to switch between vision sensors or abort the mission. During operation, the sonar is preferred for searching, because it can cover a wider area and operate at a higher altitude, whereas the camera is preferred for following and inspecting the pipeline because it is easier to detect faults in the pipeline with the camera.

3. Illustrative Examples

To use the software model for analysis with ProFeat, download the iFM 2023 artefact evaluation virtual machine (VM) [18]. In the VM, first open a terminal window. Then download the artefact⁴ with the command

```
1 wget https://zenodo.org/records/8275533/files/auv_profeat.zip
```

and unzip it with

```
1 unzip auv_profeat.zip
```

Then run the following.

³https://github.com/remaro-network/auv_profeat

⁴We will assume in the following that you saved the file in the home directory. It is also possible to save it in another directory, but then the path to this directory must be used in all commands using paths.

```
1 cd ~/prism
2 ./install.sh
```

The artefact contains a file `casestudy.profeat` with the models of the managed subsystem, the managing subsystem, and the environment, files `casestudy.fprops` and `casestudy_all.fprops` with properties to analyse, a folder `experiments` with files for conducting PRISM experiments, a license, and a README file.

3.1. Running Analyses

To run an analysis, navigate to `auv_profeat` in the terminal. Then execute the following.

```
1 ~/profeat/bin/profeat -t casestudy.profeat casestudy.fprops
2 ~/prism/bin/prism out.prism out.props > out.log
```

The first command translates the ProFeat model and the ProFeat property file to a PRISM model and property file, respectively. The second command uses PRISM to compute the results and saves them in the `out.log` file. To view the results, open the `out.log` file which is saved in the `auv_profeat` folder.

To analyse additional properties to the ones analysed here, these need to be included in the `casestudy.fprops` file, making sure to include any ProFeat-specific constructs like, e.g., features and variables, within `#{...}`.

3.2. Understanding the Output

The `out.log` file, which contains the results of the analysis, is structured as follows. After a PRISM header, it specifies the model type, the modules, and the variables of the PRISM file that was automatically translated from the ProFeat file. It then lists the analysed properties. These are slightly different from the properties specified in `casestudy.fprops` because they have been translated to PRISM properties. For each of the properties, the `out.log` file includes a paragraph, separated by `----`, with the analysis results. The result of the analysis (**Result**) can be found at the bottom of the paragraph, preceded by the time that was used for model checking (**Time for model checking**).

3.3. Changing Scenarios

The ProFeat model comes equipped with two different, predefined scenarios, each of them implementing a different behaviour of the environment. To change the scenario, open the file `~/auv_profeat/casestudy.profeat`, uncomment the parameters of the desired scenario, and comment the parameters of the other scenario. To create a new scenario, change the values of the parameters `min_visib`, `max_visib`, `current_prob`, and `inspect`. It is also possible to change the influence that the thruster failures have on the path (i.e., movement) of the AUV by changing `infl_tf`.

3.4. PRISM Experiments

It is possible to use PRISM's functionality of so-called *experiments* to perform the analysis of a parametric property for a predefined parameter range. To use this functionality with the parametric properties defined for this case study, it suffices to do as described below. The files for the PRISM experiments for the two predefined scenarios can be found in the folder `~/auv_profeat/experiments` as `scenario1.prism` and `scenario2.prism`. The necessary property file, which contains the properties used for the experiments, is `experiments.props` in the same folder.

Open the PRISM GUI via the command

```
1 ~/prism/bin/xprism
```

Open the model file of one of the two scenarios by going to **Model** → **Open model** and selecting `scenario1.prism` or `scenario2.prism`. Parse and build the model by pressing F2 and F3, respectively. To load the properties, go to the **Properties** Tab in the lower left corner. Open the properties list by going to **Properties** → **Open properties list** and select `experiments.props`. The experiments will use a variable named `k` for the number of time steps. Declare this variable by double-clicking in the empty **Constants** area, which will create an entry named `C0`. Change the name of the entry from `C0` to `k`.

To run an experiment, click one of the properties and press F7. In the dialogue that opens, first decide the desired range of parameters, i.e., how many time steps to consider. For example, to create a graph from the first property, select the radio button for **Range** (deselecting **Single Variable**), and fill in 0 for **Start**, 80 for **End** and 1 for **Step**. Then click on **Okay**, give the graph a name, and either print it to an already existing graph or to a new one.

It is also possible to inspect the values that were calculated for the graph. To do so, in the **Experiments** pane, first select (left-click) the property whose results to inspect, then right-click and select **View results** from the context menu. This will enable us to determine after how many time steps the probability for the respective property to be satisfied is above a certain threshold. In this way, we for instance determined that the probability of reaching a safe state from an unsafe state is above 0.95 after 5 time steps in both scenarios.

For more information about PRISM experiments, including how to run them from the command line, consult the PRISM manual⁵.

3.5. Model Extensions

The artefact can be modified and extended in different ways, some ideas are as follows.

- Explore new scenarios;
- Analyse different properties;
- Change the probabilities of the transitions;

⁵<https://www.prismmodelchecker.org/manual/RunningPRISM/Experiments>

- Introduce different environmental or internal parameters that can trigger adaptation (i.e., feature changes by the feature controller);
- Include new modules (e.g., introducing failures in the hardware or modelling battery consumption) that synchronise with the already existing ones and with the feature controller;
- Make the feature model richer by including more functionalities of the AUV that can be changed during runtime;
- Include new states in the AUV’s feature module (e.g., a further task that the AUV has to perform).

The latter four suggestions require extending the AUV’s feature modules, feature controller, or both.

4. Impact

The software model of the AUV case study presented in this paper is relevant for both new and existing questions in research on SASs and on dynamic SPLs as well as in industry. Below, we present the research areas in which the software model can be used and we discuss directions that highlight its relevance.

- SASs can be realised by *internal self-adaptation*, which embeds the adaptation logic in the system itself through exception handling or fault-tolerance mechanisms, or *external self-adaptation*, which separates the adaptation logic from the application logic through an external feedback loop [15, 24, 27]. Internal self-adaptation has been criticised for poor maintainability and scalability. Our software model implements external self-adaptation by separation of concerns between the application logic (the managed subsystem) and the adaptation logic (the managing subsystem) of the SAS as proposed in [15]. Thus, the software model provides an example of how to model and analyse use cases with this separation of concerns. The separation of concerns makes it easy to reuse the software model since it caters for modifications or extended use cases. Our software model also exemplifies how to improve scalability of the models by modelling all configurations and reconfigurations of an SAS in one modular model, enabling the analysis of all configurations and reconfigurations in a single run while maintaining the separation of concerns between the application and the adaptation logic.
- Dynamic SPL research distinguishes between *bounded adaptivity*, which models context variation that is anticipated at design time, and *open adaptivity*, which models context variation that is not planned at design time and requires model extension [7]. Our software model implements bounded adaptivity (the feature controller), for which dynamic SPLs have been advocated as a means to constrain the evolution of SASs, thus enabling the assessment of important properties of an SAS prior to its implementation [4].

- Dynamic SPLs have been proposed to manage runtime reconfiguration for self-adaptive robots [8, 13]. While appealing, this is still considered an unsolved challenge [12] since managing runtime reconfiguration for SASs is in general very difficult and “there is a need to validate the proposals, either in an industrial environment or in different test cases, expanding the application areas” [1]. In fact, a recent literature review [3] on testing, validation, and verification of robotic and autonomous systems does not discuss any research that uses family-based analysis techniques for SASs as exemplified with our software model. Therefore, our software model can provide an example of how to use dynamic SPLs for modelling self-adaptive robots and for using family-based analysis techniques to analyse the robot’s configurations and reconfigurations.
- Kentaro Yoshimura, chief researcher at Hitachi, recently addressed the SPL community in his keynote address entitled “The 20-year journey of SPLE in Hitachi and the next” at the 2023 SPL Conference (SPLC 2023) [17]. In his keynote, he presented the use of dynamic SPLs for autonomous robotic systems as a new industrial challenge. He said that the dynamicity is in the runtime behaviour of the autonomous robots that need to adapt and reconfigure based on input perceived from the environment without continuous human guidance. Our software model responds to this challenge by capturing the uncertainties of the environment in a separate probabilistic model that interacts with the behavioural models of the SAS.

5. Conclusions

This paper contributes a configurable software model of a dynamic SPL, reflecting the self-adaptive AUV introduced in [20]. The model is part of a growing body of family-based models including, e.g., probabilistic variants of the well-known SPL benchmarks called the Body Sensor Network (BSN) SPL [23], based on the BSN from [14], and the Elevator SPL [10], based on the Lift system from [19], both of which were analysed with ProFeat by Chrszon et al. [9] and the latter also with QFLan by ter Beek et al. [5] and its original, non-probabilistic version also with the well-known mCRL2 model-checking toolset⁶ in [6]. QFLan is a software tool for the modelling and analysis of highly reconfigurable systems, including dynamic SPLs. These software models are all publicly available.^{7,8,9} While ProFeat provides tool support for family-based (quantitative) analysis of dynamic SPLs with probabilistic behaviour through probabilistic model checking, QFLan [26] provides tool support for statistical model checking.

6. Future Plans

In the future, it would be interesting to analyse larger dynamic SPL models of SASs, which might require resorting to statistical model-checking techniques that yield statistical

⁶<https://mcrl2.org/>

⁷<https://wwwtcs.inf.tu-dresden.de/ALGI/PUB/ProFeat/>

⁸<https://github.com/qflanTeam/QFLan/wiki>

⁹<https://github.com/SjefvanLoo/VariabilityParityGames>

approximations by probabilistic simulations, thus trading 100% precision for scalability.

CRediT Author Statement

J. Päßler: Writing - Original Draft, Methodology, Software, Validation, Formal analysis, Investigation. M.H. ter Beek: Writing - Original Draft, Writing - Review & Editing, Visualization, Funding acquisition. F. Damiani: Writing - Review & Editing, Funding acquisition. E.B. Johnsen: Writing - Review & Editing, Supervision, Project administration, Funding acquisition. S.L. Tapia Tarifa: Writing - Review & Editing, Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank Clemens Dubsloff for explaining everything we always wanted to know about ProFeat. This work was supported by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200), by the Italian project NODES (which has received funding from the MUR – M4C2 1.5 of PNRR with grant agreement no. ECS00000036) and by the Italian MUR PRIN 2020TL3X8X project T-LADIES (Typeful Language Adaptation for Dynamic, Interacting and Evolving Systems).

References

- [1] Aguayo, O., Sepúlveda, S., 2022. Variability Management in Dynamic Software Product Lines for Self-Adaptive Systems—A Systematic Mapping. *Applied Sciences* 12. doi:[10.3390/app122010240](https://doi.org/10.3390/app122010240).
- [2] Apel, S., Batory, D.S., Kästner, C., Saake, G., 2013. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer. doi:[10.1007/978-3-642-37521-7](https://doi.org/10.1007/978-3-642-37521-7).
- [3] Araujo, H., Mousavi, M.R., Varshosaz, M., 2023. Testing, Validation, and Verification of Robotic and Autonomous Systems: A Systematic Review. *ACM Transactions on Software Engineering and Methodology* 32, 51:1–51:61. doi:[10.1145/3542945](https://doi.org/10.1145/3542945).
- [4] Baresi, L., 2014. Self-Adaptive Systems, Services, and Product Lines, in: *Proceedings of the 18th International Software Product Line Conference (SPLC 2014)*. ACM, pp. 2–4. doi:[10.1145/2648511.2648512](https://doi.org/10.1145/2648511.2648512).
- [5] ter Beek, M.H., Legay, A., Lluch Lafuente, A., Vandin, A., 2020a. A Framework for Quantitative Modeling and Analysis of Highly (Re)configurable Systems. *IEEE Transaction on Software Engineering* 46, 321–345. doi:[10.1109/TSE.2018.2853726](https://doi.org/10.1109/TSE.2018.2853726).
- [6] ter Beek, M.H., van Loo, S., de Vink, E.P., Willemse, T.A., 2020b. Family-Based SPL Model Checking Using Parity Games with Variability, in: Wehrheim, H., Cabot, J. (Eds.), *Proceedings of the 23rd International Conference on Fundamental Approaches to Software Engineering (FASE 2020)*. Springer. volume 12076 of *LNCS*, pp. 245–265. doi:[10.1007/978-3-030-45234-6_12](https://doi.org/10.1007/978-3-030-45234-6_12).
- [7] Bencomo, N., Hallsteinsen, S.O., de Almeida, E.S., 2012. A View of the Dynamic Software Product Line Landscape. *IEEE Computer* 45, 36–41. doi:[10.1109/MC.2012.292](https://doi.org/10.1109/MC.2012.292).
- [8] Brugali, D., Capilla, R., Hinchey, M., 2015. Dynamic Variability Meets Robotics. *IEEE Computer* 48, 94–97. doi:[10.1109/MC.2015.354](https://doi.org/10.1109/MC.2015.354).

- [9] Chrszon, P., Dubslaff, C., Klüppelholz, S., Baier, C., 2018. ProFeat: Feature-Oriented Engineering for Family-Based Probabilistic Model Checking. *Formal Aspects of Computing* 30, 45–75. doi:[10.1007/s00165-017-0432-4](https://doi.org/10.1007/s00165-017-0432-4).
- [10] Classen, A., Heymans, P., Schobbens, P.Y., Legay, A., 2011. Symbolic Model Checking of Software Product Lines, in: *Proceedings of the 33rd International Conference on Software Engineering (ICSE 2011)*. ACM, pp. 321–330. doi:[10.1145/1985793.1985838](https://doi.org/10.1145/1985793.1985838).
- [11] Cordy, M., Classen, A., Heymans, P., Legay, A., Schobbens, P.Y., 2013. Model Checking Adaptive Software with Featured Transition Systems, in: Cámara, J., de Lemos, R., Ghezzi, C., Lopes, A. (Eds.), *Assurances for Self-Adaptive Systems: Principles, Models, and Techniques*. Springer. volume 7740 of *LNCS*, pp. 1–29. doi:[10.1007/978-3-642-36249-1_1](https://doi.org/10.1007/978-3-642-36249-1_1).
- [12] García, S., Strüder, D., Brugali, D., Di Fava, A., Schillinger, P., Pelliccione, P., Berger, T., 2019. Variability Modeling of Service Robots: Experiences and Challenges, in: *Proceedings of the 13th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS 2019)*. ACM, pp. 8:1–8:6. doi:[10.1145/3302333.3302350](https://doi.org/10.1145/3302333.3302350).
- [13] Gherardi, L., Hochgeschwender, N., 2015. RRA: Models and Tools for Robotics Run-time Adaptation, in: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2015)*. IEEE, pp. 1777–1784. doi:[10.1109/IROS.2015.7353608](https://doi.org/10.1109/IROS.2015.7353608).
- [14] Hao, Y., Foster, R., 2008. Wireless body sensor networks for health-monitoring applications. *Physiological Measurement* 29, R27–R56. doi:[10.1088/0967-3334/29/11/R01](https://doi.org/10.1088/0967-3334/29/11/R01).
- [15] Kephart, J.O., Chess, D.M., 2003. The Vision of Autonomic Computing. *IEEE Computer* 36, 41–50. doi:[10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
- [16] Kwiatkowska, M., Norman, G., Parker, D., 2011. PRISM 4.0: Verification of Probabilistic Real-Time Systems, in: Gopalakrishnan, G., Qadeer, S. (Eds.), *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV 2011)*. Springer. volume 6806 of *LNCS*, pp. 585–591. doi:[10.1007/978-3-642-22110-1_47](https://doi.org/10.1007/978-3-642-22110-1_47).
- [17] Ogawa, H., Yoshimura, K., 2023. The 20-year journey of SPLE in Hitachi and the next, in: *Proceedings of the 27th ACM International Systems and Software Product Line Conference (SPLC 2023)*. ACM. volume 1, p. xix. doi:[10.1145/3579027](https://doi.org/10.1145/3579027).
- [18] Osama, M., Lamprecht, A.L., 2023. iFM 2023 artifact evaluation VM. Zenodo. doi:[10.5281/zenodo.7782241](https://doi.org/10.5281/zenodo.7782241).
- [19] Plath, M., Ryan, M., 2001. Feature integration using a feature construct. *Science of Computer Programming* 41, 53–84. doi:[10.1016/S0167-6423\(00\)00018-6](https://doi.org/10.1016/S0167-6423(00)00018-6).
- [20] Päckler, J., ter Beek, M.H., Damiani, F., Tapia Tarifa, S.L., Johnsen, E.B., 2023a. Formal Modelling and Analysis of a Self-Adaptive Robotic System, in: Herber, P., Wijs, A. (Eds.), *Proceedings of the 18th International Conference on integrated Formal Methods (iFM 2023)*. Springer. volume 14300 of *LNCS*, pp. 343–363. doi:[10.1007/978-3-031-47705-8_18](https://doi.org/10.1007/978-3-031-47705-8_18).
- [21] Päckler, J., ter Beek, M.H., Damiani, F., Tapia Tarifa, S.L., Johnsen, E.B., 2023b. Formal Modelling and Analysis of a Self-Adaptive Robotic System (Artifact). Zenodo. doi:[10.5281/zenodo.8275533](https://doi.org/10.5281/zenodo.8275533).
- [22] Rezende Silva, G., Päckler, J., Zwanepol, J., Alberts, E., Tapia Tarifa, S.L., Gerostathopoulos, I., Johnsen, E.B., Hernández Corbato, C., 2023. SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles, in: *Proceedings of the 18th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2023)*, IEEE. pp. 181–187. doi:[10.1109/SEAMS59076.2023.00031](https://doi.org/10.1109/SEAMS59076.2023.00031).
- [23] Rodrigues, G.N., Alves, V., Nunes, V., Lanna, A., Cordy, M., Schobbens, P.Y., Sharifloo, A.M., Legay, A., 2015. Modeling and Verification for Probabilistic Properties in Software Product Lines, in: *Proceedings of the 16th International Symposium on High Assurance Systems Engineering (HASE 2015)*. IEEE, pp. 173–180. doi:[10.1109/HASE.2015.34](https://doi.org/10.1109/HASE.2015.34).
- [24] Salehie, M., Tahvildari, L., 2009. Self-Adaptive Software: Landscape and Research Challenges. *ACM Transactions on Autonomous and Adaptive Systems* 4, 14:1–14:42. doi:[10.1145/1516533.1516538](https://doi.org/10.1145/1516533.1516538).
- [25] Thüm, T., Apel, S., Kästner, C., Schaefer, I., Saake, G., 2014. A Classification and Survey of Analysis Strategies for Software Product Lines. *ACM Computing Surveys* 47, 6:1–6:45. doi:[10.1145/2580950](https://doi.org/10.1145/2580950).
- [26] Vandin, A., ter Beek, M.H., Legay, A., Lluch Lafuente, A., 2018. QFLan: A Tool for the Quantitative

- Analysis of Highly Reconfigurable Systems, in: Havelund, K., Peleska, J., Roscoe, B., de Vink, E. (Eds.), Proceedings of the 22nd International Symposium on Formal Methods (FM 2018). Springer. volume 10951 of *LNCS*, pp. 329–337. doi:[10.1007/978-3-319-95582-7_19](https://doi.org/10.1007/978-3-319-95582-7_19).
- [27] Weyns, D., 2020. An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective. John Wiley & Sons. doi:[10.1002/9781119574910](https://doi.org/10.1002/9781119574910).